

Proyecto 2: QuadTree para simulación de partículas 2D

Yitzhak Abraham Namihas Millan
Carla Viviana Molina Álvarez
Sebastian Hernan Reategui Bellido

30 de junio de 2026

Resumen

El presente proyecto implementa desde cero una estructura de datos QuadTree para acelerar consultas espaciales en una simulación de partículas bidimensional. La aplicación permite insertar partículas, actualizar sus posiciones frame por frame, reconstruir el QuadTree después del movimiento, consultar regiones rectangulares, buscar vecinos cercanos y detectar posibles colisiones. Además, se compara experimentalmente el rendimiento del QuadTree contra una solución ingenua de fuerza bruta usando tres tamaños de entrada y tres distribuciones espaciales.

Índice

1. Introducción	3
2. Objetivos	3
3. Descripción de la aplicación	3
3.1. Datos de entrada configurables	3
3.2. Distribuciones implementadas	4
4. Estructura de datos QuadTree	4
4.1. Modelo de partícula	4
4.2. Invariantes	5
5. Funciones mínimas implementadas	5
6. Operaciones y complejidad	6
7. Visualización	6

8. Comparación experimental	7
8.1. Condiciones experimentales	7
8.2. Tamaños de entrada	8
8.3. Resultados	8
9. Interpretación de resultados	8
10.Evidencia visual de la aplicación	9
11.Instrucciones de ejecución	10
11.1. Backend C++	10
11.2. Frontend	10
12.Conclusiones	11
13.Archivos principales del proyecto	11

1. Introducción

El problema abordado consiste en detectar posibles colisiones entre partículas circulares que se mueven dentro de un espacio 2D. Una solución directa consiste en comparar todos los pares de partículas, lo cual requiere:

$$\frac{n(n-1)}{2}$$

comparaciones por frame. Esta estrategia tiene costo cuadrático y se vuelve ineficiente cuando aumenta la cantidad de objetos.

Para reducir la cantidad de comparaciones, se implementó un QuadTree. Esta estructura divide recursivamente el espacio 2D en cuatro regiones, permitiendo descartar zonas completas que no intersectan con una consulta espacial.

2. Objetivos

Los objetivos del proyecto son:

- Implementar correctamente una estructura de datos avanzada.
- Explicar sus invariantes, operaciones y complejidad.
- Aplicar el QuadTree a una simulación de partículas 2D.
- Construir una aplicación visual que permita observar el uso de la estructura.
- Comparar experimentalmente el QuadTree contra fuerza bruta.
- Reportar resultados para distintos tamaños y distribuciones espaciales.

3. Descripción de la aplicación

La aplicación está compuesta por un backend en C++ y un frontend web en Vue.js. El backend calcula la simulación, actualiza las partículas, construye el QuadTree y genera las métricas. El frontend muestra la simulación en tiempo real, las subdivisiones del QuadTree, la consulta rectangular, las partículas candidatas, las colisiones detectadas y las métricas de rendimiento.

3.1. Datos de entrada configurables

La interfaz permite configurar:

- Número de objetos n .
- Tamaño del espacio 2D: ancho y alto.

- Capacidad máxima por nodo del QuadTree.
- Radio de las partículas.
- Velocidad de las partículas.
- Distribución inicial de los objetos.

3.2. Distribuciones implementadas

Se implementaron tres distribuciones obligatorias:

- **Uniforme:** las partículas se distribuyen de forma aleatoria en todo el espacio.
- **Clusters:** las partículas se agrupan alrededor de varios centros.
- **Alta densidad:** una parte importante de las partículas se concentra en una zona central.

4. Estructura de datos QuadTree

Un QuadTree representa un espacio 2D mediante subdivisiones recursivas. Cada nodo contiene una región rectangular y puede almacenar partículas mientras no supere una capacidad máxima. Cuando la capacidad se excede, el nodo se divide en cuatro hijos:

- Noreste.
- Noroeste.
- Sureste.
- Suroeste.

4.1. Modelo de partícula

Cada partícula tiene un identificador, posición, velocidad y radio:

```
1 struct Particle {  
2     int id;  
3     double x, y;  
4     double vx, vy;  
5     double radius;  
6 };
```

4.2. Invariantes

Durante la ejecución se mantienen los siguientes invariantes:

1. Toda partícula insertada en un nodo hijo pertenece espacialmente a los límites de dicho hijo.
2. Un nodo hoja contiene como máximo la capacidad configurada.
3. Si un nodo supera la capacidad máxima, se subdivide en exactamente cuatro regiones.
4. Los nodos internos delegan la inserción de partículas a sus hijos.
5. La consulta espacial solo recorre nodos cuya región intersecta con la región consultada.

5. Funciones mínimas implementadas

La Tabla 1 resume las funcionalidades requeridas y dónde se encuentran en el código.

Tabla 1: Funciones mínimas del proyecto

Requisito	Implementación
Insertar objetos en el QuadTree	Función <code>QuadNode::insert</code> en <code>Structure.cpp</code> .
Subdividir regiones por capacidad	Función <code>QuadNode::subdivide</code> en <code>Structure.cpp</code> .
Actualizar posiciones frame por frame	Función <code>updateParticles</code> en <code>Structure.cpp</code> .
Reconstruir el QuadTree después del movimiento	Se realiza en cada frame antes de consultar colisiones y candidatos.
Consultar objetos en región rectangular	Función <code>queryRange</code> ; se visualiza mediante un rectángulo en el canvas.
Consultar objetos cercanos a un punto	Función <code>queryNearPoint</code> .
Detectar posibles colisiones	Función <code>quadTreeCollisionStats</code> .
Comparar contra fuerza bruta	Función <code>bruteForceCollisionStats</code> y módulo <code>Experiment.cpp</code> .

Como verificación de consistencia, la implementación experimental compara en cada frame las colisiones encontradas por el QuadTree contra las obtenidas por fuerza bruta. Si ambos métodos no coinciden, el programa muestra una advertencia en consola. Esta comprobación es importante porque confirma que la reducción de comparaciones no cambia el resultado final de detección.

6. Operaciones y complejidad

Tabla 2: Complejidad esperada de las operaciones

Operación	Caso promedio	Peor caso
Inserción	$O(\log n)$	$O(n)$
Subdividir nodo	$O(c)$; si c es constante, se considera $O(1)$	$O(c)$
Consulta rectangular	$O(\log n + k)$	$O(n)$
Consulta por cercanía	$O(\log n + k)$	$O(n)$
Detección con Quad-Tree	$O(n \log n)$	$O(n^2)$
Fuerza bruta	$O(n^2)$	$O(n^2)$

Donde k representa la cantidad de objetos candidatos retornados por una consulta y c representa la capacidad máxima de partículas por nodo.

Un caso límite que debe controlarse en una versión final es la inserción de muchas partículas con la misma coordenada o dentro de una región extremadamente pequeña. En ese escenario, el QuadTree puede subdividirse demasiadas veces. Para evitarlo, se recomienda fijar una profundidad máxima o un tamaño mínimo de celda; cuando se alcance ese límite, las partículas restantes pueden conservarse en el nodo hoja.

7. Visualización

La visualización muestra:

- Partículas en el plano.
- Subdivisiones actuales del QuadTree.
- Región rectangular consultada.
- Objetos candidatos retornados por el QuadTree.
- Colisiones detectadas.
- Número de comparaciones por QuadTree y fuerza bruta.
- Tiempo por frame.

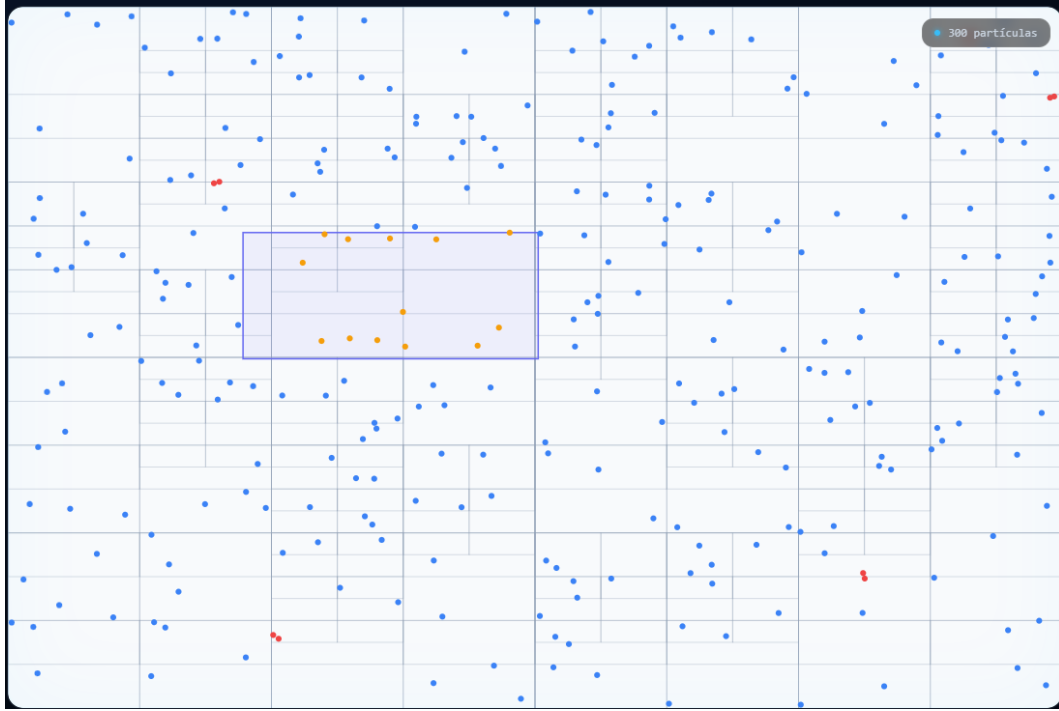


Figura 1: Simulación visual con partículas, subdivisiones del QuadTree y consulta rectangular.

8. Comparación experimental

La comparación experimental se realizó ejecutando el QuadTree y la solución de fuerza bruta sobre los mismos datos. Para cada caso se midieron:

- Tiempo promedio de construcción del QuadTree.
- Tiempo promedio de detección con QuadTree.
- Tiempo promedio de detección con fuerza bruta.
- Número promedio de comparaciones.
- Número promedio de candidatos por partícula.
- Cantidad promedio de colisiones.
- Speedup obtenido respecto a fuerza bruta.

8.1. Condiciones experimentales

Cada experimento se ejecutó durante 30 frames por caso, usando un espacio de simulación de 1000x1000. Para cada combinación de tamaño y distribución se registraron los tiempos promedio de construcción, detección con QuadTree y detección con fuerza bruta. También se registraron las comparaciones realizadas, los candidatos promedio por partícula, las colisiones detectadas y el speedup obtenido.

El speedup presentado en la tabla se calculó sobre el tiempo de detección, es decir, $T. BF/T. QT$. Como el QuadTree se reconstruye en cada frame, para analizar el costo total por frame también debe considerarse $Build QT + T. QT$. Esta aclaración evita confundir el tiempo de detección con el tiempo completo de simulación.

8.2. Tamaños de entrada

Se usaron tres tamaños de simulación:

- 1,000 partículas.
- 5,000 partículas.
- 10,000 partículas.

Estos tamaños permiten observar cómo cambia el rendimiento al aumentar la cantidad de datos.

8.3. Resultados

La Tabla 3 presenta los resultados obtenidos en la pestaña de experimentos de la aplicación.

Tabla 3: Resultados experimentales QuadTree vs fuerza bruta

n	Distribución	Build QT	T. QT	T. BF	Comp. QT	Comp. BF	Cand./Part.	Colisiones	Speedup
1000	Uniforme	266 μs	423 μs	2.45 ms	128	499,500	0.13	100	5.8x
1000	Clusters	267 μs	619 μs	2.57 ms	685	499,500	0.69	535	4.2x
1000	Alta densidad	261 μs	765 μs	2.52 ms	1,335	499,500	1.34	1,054	3.3x
5000	Uniforme	1.58 ms	3.46 ms	61.94 ms	3,223	12,497,500	0.64	2,532	17.9x
5000	Clusters	1.62 ms	6.58 ms	62.97 ms	18,638	12,497,500	3.73	14,637	9.6x
5000	Alta densidad	1.61 ms	8.24 ms	62.62 ms	33,207	12,497,500	6.64	26,093	7.6x
10000	Uniforme	3.43 ms	9.22 ms	253.42 ms	12,838	49,995,000	1.28	10,102	27.5x
10000	Clusters	3.74 ms	20.10 ms	261.89 ms	73,205	49,995,000	7.32	57,521	13.0x
10000	Alta densidad	99.85 ms	25.31 ms	264.06 ms	132,958	49,995,000	13.30	104,503	10.4x

9. Interpretación de resultados

Para analizar los experimentos se consideraron tres tamaños de entrada: 1,000, 5,000 y 10,000 partículas, evaluados bajo tres distribuciones espaciales: uniforme, clusters y alta densidad. En la tabla, **Build QT** representa el tiempo de construcción del QuadTree; **T. QT**, el tiempo usado por el QuadTree para detectar posibles colisiones; y **T. BF**, el tiempo usado por fuerza bruta. Asimismo, **Comp. QT** y **Comp. BF** indican la cantidad de comparaciones realizadas por cada método, mientras que **Cand./Part.** muestra el promedio de candidatos revisados por partícula. **Colisiones** corresponde a los encuentros detectados y **Speedup** indica cuántas veces más rápido fue el QuadTree respecto a fuerza bruta.

Los resultados muestran que fuerza bruta realiza siempre $\frac{n(n-1)}{2}$ comparaciones, porque compara todos los pares posibles de partículas. Por ello, al aumentar el número de objetos, su costo crece de forma cuadrática. En cambio, el QuadTree reduce las comparaciones al

dividir el espacio en regiones y revisar solo las partículas cercanas o relevantes para cada consulta.

La distribución uniforme fue el caso más favorable para el QuadTree. Con 10,000 partículas, fuerza bruta realizó 49,995,000 comparaciones, mientras que el QuadTree solo realizó 12,838. Esto permitió reducir el tiempo de 253.42 ms a 9.22 ms, alcanzando un speedup de 27.5x. Esto ocurre porque las partículas están mejor repartidas y el QuadTree puede descartar muchas regiones durante la búsqueda.

En los casos de clusters y alta densidad, el rendimiento del QuadTree disminuyó, aunque siguió siendo mejor que fuerza bruta. Esto se debe a que muchas partículas se concentran en zonas pequeñas, aumentando los candidatos por partícula. Por ejemplo, con 10,000 partículas, la distribución uniforme tuvo 1.28 candidatos por partícula, mientras que clusters tuvo 7.32 y alta densidad 13.30. Por esta razón, el speedup bajó a 13.0x y 10.4x respectivamente.

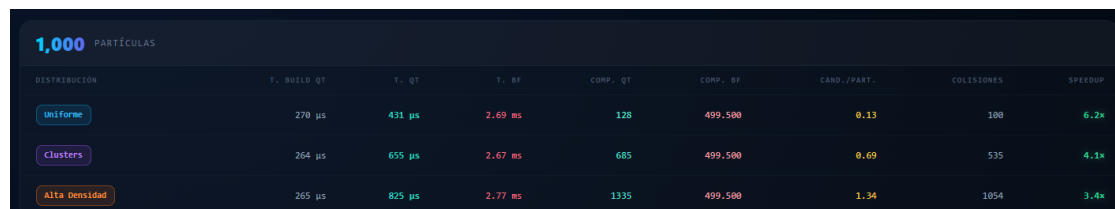
El caso de alta densidad con 10,000 partículas presentó el mayor tiempo de construcción del QuadTree. Este resultado puede explicarse por la concentración de partículas en una zona reducida, lo que obliga a generar más subdivisiones y aumenta el costo de inserción dentro de la estructura.

Si se considera el costo total por frame, el caso uniforme con 10,000 partículas pasa de 253,42 ms en fuerza bruta a $3,43 + 9,22 = 12,65$ ms con QuadTree, lo que mantiene una mejora aproximada de 20,0x. En alta densidad con 10,000 partículas, el costo total del QuadTree es $99,85 + 25,31 = 125,16$ ms frente a 264,06 ms de fuerza bruta, con una mejora aproximada de 2,1x. Por ello, el QuadTree sigue siendo favorable, pero la construcción del árbol debe reportarse de forma separada para interpretar correctamente el rendimiento.

En conclusión, el QuadTree demostró ser más eficiente que fuerza bruta para la detección de posibles colisiones en una simulación 2D. Su ventaja es mayor cuando los datos están distribuidos de forma uniforme, pero incluso en escenarios con alta concentración de partículas mantiene una reducción importante en comparaciones y tiempo de ejecución.

10. Evidencia visual de la aplicación

En esta sección se muestran capturas de la aplicación ejecutándose, incluyendo la simulación de partículas, las subdivisiones del QuadTree, la consulta rectangular, los candidatos detectados y los resultados experimentales.



DISTRIBUCION	T. BUILD QT	T. QT	T. BF	COMP. QT	COMP. BF	CAND./PART.	COLISIONES	SPEEDUP
Uniforme	270 μ s	431 μ s	2.69 ms	128	499,500	0.13	100	6.2x
Clusters	264 μ s	655 μ s	2.67 ms	685	499,500	0.69	535	4.1x
Alta Densidad	265 μ s	825 μ s	2.77 ms	1335	499,500	1.34	1054	3.4x

Figura 2: Resultados experimentales para 1,000 partículas.

5,000 PARTÍCULAS								
DISTRIBUCIÓN	T. BUILD QT	T. QT	T. BF	COMP. QT	COMP. BF	CAND./PART.	COLISIONES	SPEEDUP
Uniforme	1.67 ms	3.78 ms	68.94 ms	3223	12,497,500	0.64	2532	18.2x
Clusters	1.59 ms	7.03 ms	66.63 ms	18,638	12,497,500	3.73	14,637	9.5x
Alta Densidad	1.61 ms	8.79 ms	67.23 ms	33,287	12,497,500	6.64	26,093	7.6x

Figura 3: Resultados experimentales para 5,000 partículas.

10,000 PARTÍCULAS								
DISTRIBUCIÓN	T. BUILD QT	T. QT	T. BF	COMP. QT	COMP. BF	CAND./PART.	COLISIONES	SPEEDUP
Uniforme	3.31 ms	9.39 ms	267.21 ms	12,838	49,995,000	1.28	10,102	28.4x
Clusters	3.46 ms	146.75 ms	141.73 ms	73,205	49,995,000	7.32	57,521	1.0x
Alta Densidad	3.23 ms	25.47 ms	262.44 ms	132,958	49,995,000	13.30	104,503	10.3x

Figura 4: Resultados para 10,000 partículas y visualización comparativa de comparaciones realizadas por el QuadTree.

1,000 PARTÍCULAS – MAX SPEEDUP		5,000 PARTÍCULAS – MAX SPEEDUP		10,000 PARTÍCULAS – MAX SPEEDUP	
Uniforme	6.2x	Uniforme	18.2x	Uniforme	28.4x
Clusters	4.1x	Clusters	9.5x	Clusters	1.0x
Alta Densidad	3.4x	Alta Densidad	7.6x	Alta Densidad	10.3x

Figura 5: Resumen de speedup máximo por tamaño de entrada y distribución espacial.

11. Instrucciones de ejecución

11.1. Backend C++

Antes de compilar, el archivo `httplib.h` debe estar en la misma carpeta del backend o en una ruta incluida por el compilador. En Linux se recomienda compilar con optimización y soporte de hilos:

```

1 g++ -std=c++20 -O2 -pthread Main.cpp Structure.cpp Visualization.
  cpp Experiment.cpp -o server
2 ./server

```

11.2. Frontend

```

1 cd frontend
2 npm install
3 npm run dev

```

Luego abrir en el navegador:

<http://localhost:5173>

12. Conclusiones

El proyecto cumple con la implementación de un QuadTree desde cero y su aplicación en una simulación de partículas 2D. La estructura permitió reducir significativamente el número de comparaciones frente a la fuerza bruta, especialmente en la distribución uniforme. Por ejemplo, con 10,000 partículas, la fuerza bruta realizó 49,995,000 comparaciones, mientras que el QuadTree realizó solo 12,838 comparaciones durante la detección, alcanzando un speedup de detección de 27.5x.

Los resultados también muestran que la distribución espacial influye directamente en el rendimiento. Cuando las partículas están distribuidas uniformemente, el QuadTree descarta más regiones y revisa menos candidatos. En cambio, en clusters y alta densidad, el número de candidatos por partícula aumenta, lo que reduce el speedup. Aun así, incluso en el caso de alta densidad con 10,000 partículas, el QuadTree logró ser más rápido que fuerza bruta durante la detección. Al considerar también la construcción por frame, la mejora total se reduce, pero sigue siendo favorable.

Finalmente, la aplicación visual permite observar claramente los elementos principales del algoritmo: las partículas, las subdivisiones del QuadTree, la región consultada, los candidatos retornados, las colisiones detectadas y las métricas de comparación. Esto demuestra que el QuadTree mejora el rendimiento y permite entender visualmente cómo se filtran las regiones relevantes durante la simulación.

13. Archivos principales del proyecto

- `Structure.cpp` / `Structure.h`: implementación del QuadTree, consultas, colisiones y fuerza bruta.
- `Experiment.cpp` / `Experiment.h`: generación de distribuciones y benchmarks.
- `Visualization.cpp` / `Visualization.h`: generación de frames, consulta visual y serialización.
- `Main.cpp`: servidor C++ con endpoints REST y SSE.
- `frontend/src/App.vue`: interfaz principal.
- `frontend/src/components/QuadCanvas.vue`: renderizado de partículas, subdivisiones y consulta rectangular.